



**MALAD KANDIVALI EDUCATION SOCIETY'S
NAGINDAS KHANDWALA COLLEGE OF COMMERCE,
ARTS & MANAGEMENT STUDIES & SHANTABEN NAGINDAS
KHANDWALA COLLEGE OF SCIENCE
MALAD [W], MUMBAI – 64
(AUTONOMOUS)
(Reaccredited 'A' Grade by NAAC)
(AFFILIATED TO UNIVERSITY OF MUMBAI)
(ISO 9001:2015)**

CERTIFICATE

Name:

Roll No:

Programme: Advanced Data Structure

Semester: I

This is certified to be a bonafide record of practical works done by the above student in the college laboratory for the course Advanced Data Structure for the partial fulfilment of Semester I of MSc CS during the academic year 2026-2027.

The journal work is the original study work that has been duly approved in the year 2026-2027 by the undersigned.

External Examiner

(Subject-In-Charge)

Prof. Ankit Javeri

Date of Examination

(College Stamp)

INDEX

SUBJECT: ADVANCED DATA STRUCTURE PRACTICALS

CLASS: FY MSC CS

SEMESTER: I

ROLL NO:

SR NO.	DATE	PRACTICAL AIM	SIGN
1	04-07-26	Implement Hash Function	
2	11-07-26	Build Array Dictionary	
3	18-07-26	Implement Linear Probing	
4	25-07-26	Implement Quadratic Probing	
5	01-08-26	Implement Double Hashing	
6	08-08-26	Implement Separate Chaining	
7	22-08-26	Perform Dynamic Rehashing	
8	22-08-26	Implement Dictionary Operations	

PRACTICAL NO.1

AIM: Implement Hash Function

CODE:

```
1 | S, T = 3, [[] for _ in range(3)]
2 | print("Empty:", T)
3 |
4 | def insert(k): T[k % S].append(k); print(f"Insert {k}:", T)
5 | def lookup(k): print(f"Lookup {k}:", k in T[k % S])
6 | def delete(k):
7 |     if k in T[k % S]: T[k % S].remove(k)
8 |     print(f"Delete {k}:", T)
9 | def update(o, n): delete(o); insert(n)
10 |
11 | insert(10)
12 | insert(13)
13 | lookup(13)
14 | update(10, 16)
15 | delete(13)
```

OUTPUT:

```
Empty: [[], [], []]
Insert 10: [[], [10], []]
Insert 13: [[], [10, 13], []]
Lookup 13: True
Delete 10: [[], [13], []]
Insert 16: [[], [13, 16], []]
Delete 13: [[], [16], []]
```

PRACTICAL NO.2

AIM: Build Array Dictionary

CODE:

```
1 S, T = 5, [[] for _ in range(5)]
2 b = lambda k: T[hash(k) % S]
3 def insert(k, v):
4     b(k)[:] = [x for x in b(k) if x[0] != k] + [[k, v]]
5     print(f"Insert {k}:", T)
6 def delete(k):
7     b(k)[:] = [x for x in b(k) if x[0] != k]
8     print(f"Delete {k}:", T)
9 lookup = lambda k: print(f"Lookup {k}:", dict(b(k)).get(k))
10 update = insert
11 insert("cat", 1)
12 lookup("cat")
13 update("cat", 10)
14 delete("cat")
```

OUTPUT:

```
Insert cat: [[], [['cat', 1]], [], [], []]
Lookup cat: 1
Insert cat: [[], [['cat', 10]], [], [], []]
Delete cat: [[], [], [], [], []]
```

PRACTICAL NO.3

AIM: Implement Linear Probing

CODE:

```
1 S, T = 7, [None] * 7
2 print("Initial Table:", T)
3
4 def insert(key):
5     print(f"\n--- Inserting Key: {key} ---")
6     for probe in range(S):
7         idx = (key % S + probe) % S
8         if T[idx] is None:
9             T[idx] = key
10            print(f"  Probe {probe}: Slot {idx} is FREE -> Placed {key} at index {idx}")
11            print(f"  Table State: {T}")
12            return
13            print(f"  Probe {probe}: Slot {idx} Has {T[idx]} -> Collision for key {key}")
14
15 for key in [14, 21, 28]:
16     insert(key)
```

OUTPUT:

```
Initial Table: [None, None, None, None, None, None, None]

--- Inserting Key: 14 ---
  Probe 0: Slot 0 is FREE -> Placed 14 at index 0
  Table State: [14, None, None, None, None, None, None]

--- Inserting Key: 21 ---
  Probe 0: Slot 0 Has 14 -> Collision for key 21
  Probe 1: Slot 1 is FREE -> Placed 21 at index 1
  Table State: [14, 21, None, None, None, None, None]

--- Inserting Key: 28 ---
  Probe 0: Slot 0 Has 14 -> Collision for key 28
  Probe 1: Slot 1 Has 21 -> Collision for key 28
  Probe 2: Slot 2 is FREE -> Placed 28 at index 2
  Table State: [14, 21, 28, None, None, None, None]
```

PRACTICAL NO.4

AIM: Implement Quadratic Probing

CODE:

```
1 | S, T = 11, [None] * 11
2 |
3 | def insert(k):
4 |     for i in range(S):
5 |         if T[x := (k % S + i**2) % S] is None:
6 |             T[x] = k
7 |             return print(f"Probe {i}: Placed {k} at slot {x} -> {T}")
8 |             print(f"Probe {i}: Collision for {k} at slot {x} with {T[x]}")
9 |
10 | for k in [22, 33, 44]:
11 |     insert(k)
```

OUTPUT:

```
Probe 0: Placed 22 at slot 0 -> [22, None, None, None, None, None, None, None, None, None, None]
Probe 0: Collision for 33 at slot 0 with 22
Probe 1: Placed 33 at slot 1 -> [22, 33, None, None, None, None, None, None, None, None, None]
Probe 0: Collision for 44 at slot 0 with 22
Probe 1: Collision for 44 at slot 1 with 33
Probe 2: Placed 44 at slot 4 -> [22, 33, None, None, 44, None, None, None, None, None, None]
```

PRACTICAL NO.5

AIM: Implement Double Hashing

CODE:

```
1 | S, P, T = 13, 7, [None] * 13
2 | def insert(k):
3 |     h1, h2 = k % S, P - (k % P)
4 |     for i in range(S):
5 |         if T[x := (h1 + i * h2) % S] is None:
6 |             T[x] = k
7 |             return print(f"Probe {i}: Placed {k} at slot {x} (h2={h2}) -> {T}")
8 |         print(f"Probe {i}: Collision for {k} at slot {x} with {T[x]}")
9 |
10 | for k in [26, 39, 52]:
11 |     insert(k)
```

OUTPUT:

Probe 0: Placed 26 at slot 0 (h2=2) -> [26, None, None, None, None, None, None, None, None, None, None, None, None]
Probe 0: Collision for 39 at slot 0 with 26
Probe 1: Placed 39 at slot 3 (h2=3) -> [26, None, None, 39, None, None, None, None, None, None, None, None, None]
Probe 0: Collision for 52 at slot 0 with 26
Probe 1: Placed 52 at slot 4 (h2=4) -> [26, None, None, 39, 52, None, None, None, None, None, None, None, None]

PRACTICAL NO.6

AIM: Implement Separate Chaining

CODE:

```
1 | s, T = 17, [[] for _ in range(17)]
2 |
3 | def insert(k):
4 |     b = T[x := k % s]
5 |     c = f"Collision #{len(b)}" if b else "HEAD"
6 |     b.append(k)
7 |     print(f"Key {k:2d} -> Bucket {x:2d} [{c:12s}]: {b}")
8 |
9 | for k in [5, 22, 10, 27, 44, 3]:
10 |     insert(k)
```

OUTPUT:

```
Key  5 -> Bucket  5 [HEAD           ]: [5]
Key 22 -> Bucket  5 [Collision #1]: [5, 22]
Key 10 -> Bucket 10 [HEAD           ]: [10]
Key 27 -> Bucket 10 [Collision #1]: [10, 27]
Key 44 -> Bucket 10 [Collision #2]: [10, 27, 44]
Key  3 -> Bucket  3 [HEAD           ]: [3]
```

PRACTICAL NO.7

AIM: Perform Dynamic Rehashing

CODE:

```
1|old_T, N = [5, 12, 19], 7
2|new_T = [None] * N
3|
4|def rehash(k):
5|    for i in range(N):
6|        if new_T[x := (k % N + i) % N] is None:
7|            new_T[x] = k
8|            return print(f"Probe {i}: Placed {k} at slot {x} -> {new_T}")
9|            print(f"Probe {i}: Collision for {k} at slot {x} with {new_T[x]}")
10|
11|for k in old_T:
12|    rehash(k)
```

OUTPUT:

```
Probe 0: Placed 5 at slot 5 -> [None, None, None, None, None, 5, None]
Probe 0: Collision for 12 at slot 5 with 5
Probe 1: Placed 12 at slot 6 -> [None, None, None, None, None, 5, 12]
Probe 0: Collision for 19 at slot 5 with 5
Probe 1: Collision for 19 at slot 6 with 12
Probe 2: Placed 19 at slot 0 -> [19, None, None, None, None, 5, 12]
```

PRACTICAL NO.8

AIM: Implement Dictionary Operations

CODE:

```
1 S, T, D = 7, [None] * 7, "DEL"
2 def op(act, k, v=None, ts=None):
3     print(f"\n{act.upper()} key={k}:")
4     for i in range(S):
5         e = T[x := (k % S + i) % S]
6         print(f"Probe {i} (slot {x}): {e}")
7         ts = x if act == "ins" and e == D and ts is None else ts
8         if (m := type(e) is list and e[0] == k) or e is None:
9             if m: T[x] = D if act == "del" else [k, v] if act != "get" else e
10            elif act in ("ins", "up"): T[x if ts is None else ts] = [k, v]
11            return print(" Result:", T)
12
13 tasks = [("ins", 10, "red"), ("ins", 17, "blue"), ("up", 17, "green"), ("del", 10), ("get", 17)]
14 for t in tasks:
15     op(*t)
```

OUTPUT:

```
INS key=10:
Probe 0 (slot 3): None
Result: [None, None, None, [10, 'red'], None, None, None]

INS key=17:
Probe 0 (slot 3): [10, 'red']
Probe 1 (slot 4): None
Result: [None, None, None, [10, 'red'], [17, 'blue'], None, None]

UP key=17:
Probe 0 (slot 3): [10, 'red']
Probe 1 (slot 4): [17, 'blue']
Result: [None, None, None, [10, 'red'], [17, 'green'], None, None]

DEL key=10:
Probe 0 (slot 3): [10, 'red']
Result: [None, None, None, 'DEL', [17, 'green'], None, None]

GET key=17:
Probe 0 (slot 3): DEL
Probe 1 (slot 4): [17, 'green']
Result: [None, None, None, 'DEL', [17, 'green'], None, None]
```